

Problem Statement :

Implement FCFS CPU Scheduling for the following scenario where No of processes = 5 and Process informations are-

P id	AT	BT
P0	3	1
P1	5	3
P2	2	2
P3	1	2
P4	6	3

Calculate CT, TAT, WT, AVG TAT and AVG WT. Also represent the process execution sequence.

Source Code :

```
p_id = ["p0", "p1", "p2", "p3", "p4"]
```

```
at = [3, 5, 2, 1, 6]
```

```
bt = [1, 3, 2, 2, 3]
```

```
for i in range(len(at) - 1):
```

```
    for j in range(len(at) - 1 - i):
```

```
        if at[j] > at[j + 1]:
```

```
            temp = at[j]
```

```
            at[j] = at[j + 1]
```

```
            at[j + 1] = temp
```

```
temp = bt[j]
```

```
bt[j] = bt[j + 1]
```

```
bt[j + 1] = temp
```

```
temp = p_id[j]
```

```
p_id[j] = p_id[j + 1]
```

```

        p_id[j + 1] = temp
ct = []
tat = []
wt = []
time = 0
sum_tat = 0
sum_wt = 0

for i in range(len(p_id)):
    if time < at[i]:
        t = at[i]

        ct.append(t + bt[i])
        t = ct[i]

        tat.append(ct[i] - at[i])
        wt.append(tat[i] - bt[i])

        sum_tat += tat[i]
        sum_wt += wt[i]
print("p_id\AT\BT\CT\TAT\WT")

for i in range(len(p_id)):
    print(p_id[i], "\t", at[i], "\t", bt[i], "\t", ct[i], "\t", wt[i])

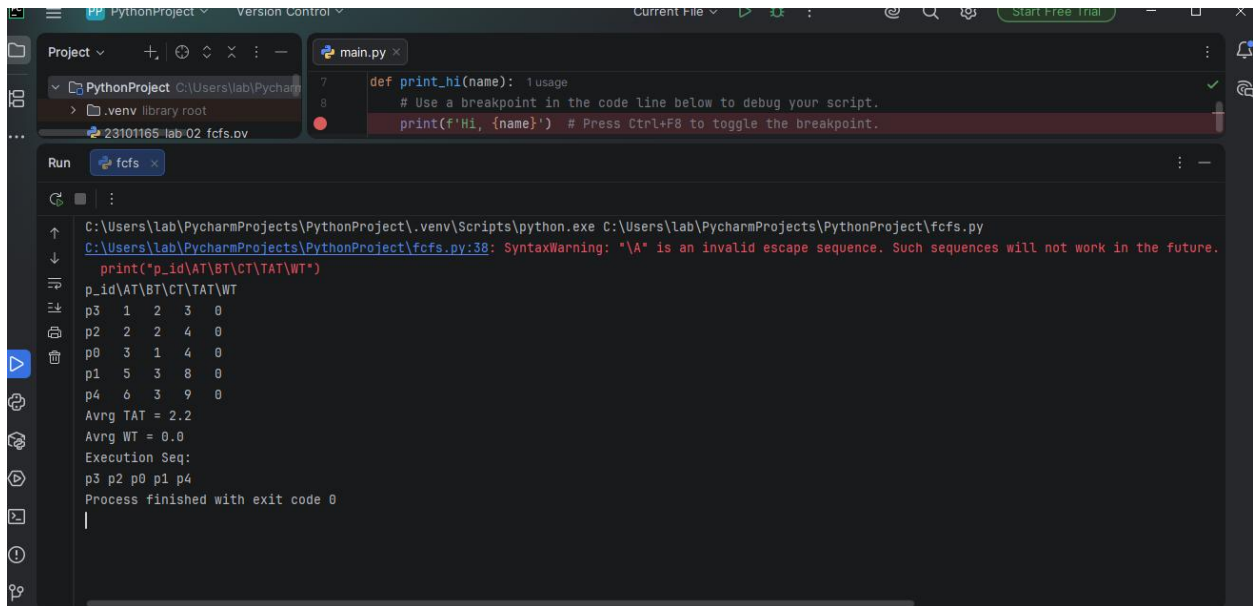
print("Avrg TAT =", sum_tat / len(p_id))
print("Avrg WT =", sum_wt / len(p_id))

print("Execution Seq:")

```

```
for i in range(len(p_id)):
    print(p_id[i], end=" ")
```

Output:



The screenshot shows the PyCharm IDE interface. The top pane displays a Python script named `main.py` with the following code:

```
def print_hi(name):
    # Use a breakpoint in the code line below to debug your script.
    print(f'Hi, {name}') # Press Ctrl+F8 to toggle the breakpoint.
```

The bottom pane shows the output of the script, which includes a syntax warning and the following data:

```
C:\Users\lab\PycharmProjects\PythonProject\venv\Scripts\python.exe C:\Users\lab\PycharmProjects\PythonProject\fcfs.py
C:\Users\lab\PycharmProjects\PythonProject\fcfs.py:38: SyntaxWarning: "\A" is an invalid escape sequence. Such sequences will not work in the future.
  print("p_id\AT\BT\CT\TAT\WT")
p_id\AT\BT\CT\TAT\WT
p3 1 2 3 0
p2 2 2 4 0
p0 3 1 4 0
p1 5 3 8 0
p4 6 3 9 0
Avg TAT = 2.2
Avg WT = 0.0
Execution Seq:
p3 p2 p0 p1 p4
Process finished with exit code 0
```

Discussion :

While implementing this program, I encountered several challenges and resolved them through trial, debugging, and logical analysis.

The first challenge was sorting the processes based on their Arrival Time. Initially, I tried to use Python's built-in sorting method with a lambda expression. However, I was not familiar with how the lambda function works, so I found it difficult to implement correctly. To overcome this problem, I decided to use the Bubble Sort algorithm because it is simple, easy to understand, and I was more comfortable implementing it manually. While sorting the Arrival Time array, I also swapped the corresponding Burst Time and Process ID values to maintain the correct relationship among the process information.

Another issue I faced was related to syntax and variable naming. At first, I used the variable `t = 0` to represent the current execution time. Later, I reused the same variable for another purpose, which created confusion in the program and resulted in errors. After identifying the problem, I replaced the variable with `time = 0`, making its purpose

clearer and avoiding variable conflicts. This modification resolved the errors, and the program executed successfully.

In addition, I carefully verified the calculations of Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT). I tested the program multiple times, corrected logical mistakes, and ensured that each calculation produced the expected output. Through continuous debugging and testing, I was able to generate the correct execution sequence, average turnaround time, and average waiting time.

Conclusion:

This assignment provided me with practical experience in implementing a CPU Scheduling algorithm using Python. Although I faced several difficulties during the development process, such as understanding the lambda function, implementing the sorting logic, and resolving variable-related errors, I successfully solved these problems through debugging and logical analysis.

Instead of relying on advanced Python features, I implemented the sorting process using the Bubble Sort algorithm, which helped me better understand how process scheduling works. I also learned the importance of using meaningful variable names and carefully tracing program execution to identify and fix errors.

Overall, this assignment improved my programming, debugging, and problem-solving skills. It also strengthened my understanding of process scheduling concepts and gave me confidence to solve similar programming problems more efficiently in the future.